CMSC132: Sets and Maps

By Naren Ayinala

May 2026

Contents

1 Why Sets and Maps Exist **2**

2 Sets: No Duplicate Elements **2**
2.1 What does uniqueness mean? 2
2.2 Set operations: union, intersection, difference 2
2.3 HashSet, LinkedHashSet, and TreeSet 3

3 Maps: Keys and Values **3**
3.1 What if a key is absent? 3
3.2 Three views of a map 4
3.3 HashMap, LinkedHashMap, TreeMap, Hashtable 4

4 The equals/hashCode Connection **4**

5 A Complete Example: Course Enrollment **4**

6 Common Mistakes and Exam Traps **5**

7 Practice Questions with Answers **5**

1 Why Sets and Maps Exist

Imagine that we are building a program to track which students attended a review session. If we add the same student twice, should we count two attendees? No! We want a **Set**: a collection of distinct elements. Now imagine recording each student's score. We want to look up a score by a student ID. That is a **Map**: a collection of key–value associations.

Both `Set<E>` and `Map<K,V>` are **interfaces**, not concrete classes. In Java, you choose an implementation such as `HashSet`, `TreeSet`, `HashMap`, or `TreeMap`. This is exactly the interface-versus-implementation distinction you learned earlier in CMSC132.

```
Set<String> attendees = new HashSet<>();
Map<String, Integer> scores = new HashMap<>();
```

A map is not a subtype of `Collection`; its entries associate keys with values rather than representing individual collection elements. Both interfaces are in `java.util`.

2 Sets: No Duplicate Elements

2.1 What does uniqueness mean?

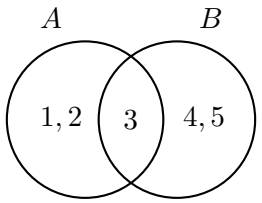
For a `HashSet`, membership is determined using `hashCode()` and `equals()`; for a `TreeSet`, its ordering comparison determines whether elements are considered duplicates. This matters when you store your own classes, not just strings and integers.

```
Set<String> names = new HashSet<>();
System.out.println(names.add("Naren")); // true
System.out.println(names.add("Naren")); // false
System.out.println(names.size());        // 1
System.out.println(names.contains("Naren")); // true
System.out.println(names.remove("Naren"));   // true
```

The second `add` returns false because that element was already present. Notice that `Set` does not offer `get(0)`: a set is not an indexed sequence.

2.2 Set operations: union, intersection, difference

Mathematically, union collects everything in either set; intersection collects only common elements; and difference keeps elements from the first set that are not in the second. In Java these operations are performed with `addAll`, `retainAll`, and `removeAll`, respectively. There are no standard `union()` or `intersection()` methods on `Set`.



Here $A = \{1, 2, 3\}$ and $B = \{3, 4, 5\}$. Therefore $A \cup B = \{1, 2, 3, 4, 5\}$, $A \cap B = \{3\}$, and $A - B = \{1, 2\}$.

```
Set<Integer> a = new HashSet<>(Arrays.asList(1, 2, 3));
Set<Integer> b = new HashSet<>(Arrays.asList(3, 4, 5));
Set<Integer> union = new HashSet<>(a);
union.addAll(b);
Set<Integer> intersection = new HashSet<>(a);
intersection.retainAll(b);
```

```
Set<Integer> difference = new HashSet<>(a);
difference.removeAll(b);
// a and b are unchanged because we copied a first.
```

2.3 HashSet, LinkedHashSet, and TreeSet

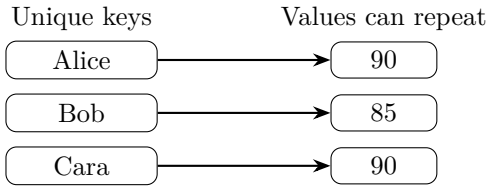
Implementation	Iteration order	Typical add/contains/remove
`HashSet`	No guaranteed order	$O(1)$ average
`LinkedHashSet`	Insertion order	$O(1)$ average
`TreeSet`	Sorted order	$O(\log n)$

```
Set<Integer> sorted = new TreeSet<>();
sorted.add(40); sorted.add(10); sorted.add(30);
System.out.println(sorted); // [10, 30, 40]
```

A `TreeSet` normally uses natural ordering (`Comparable`) or a supplied `Comparator`. A `HashSet` does not sort values. `LinkedHashSet` retains insertion order, not sorted order.

3 Maps: Keys and Values

A map stores *one value per unique key*. Different keys may share the same value. Inserting a key that already exists *replaces* its old value rather than creating a second copy of the key.




```
Map<String, Integer> grades = new HashMap<>();
grades.put("Alice", 90);
grades.put("Bob", 85);
grades.put("Alice", 95); // replaces 90
System.out.println(grades.get("Alice")); // 95
System.out.println(grades.size());       // 2
System.out.println(grades.containsKey("Bob")); // true
System.out.println(grades.containsValue(95)); // true
System.out.println(grades.remove("Bob")); // 85
```

3.1 What if a key is absent?

`get(key)` returns `null` if there is no mapping, but a `HashMap` can also deliberately map a key to `null`. To distinguish these cases use `containsKey`. For simple numeric counts, `getOrDefault` is convenient.

```
Map<String, Integer> counts = new HashMap<>();
for (String word : Arrays.asList("cat", "dog", "cat")) {
    counts.put(word, counts.getOrDefault(word, 0) + 1);
}
System.out.println(counts.get("cat")); // 2
```

3.2 Three views of a map

`keySet()` gives the keys, `values()` gives the values, and `entrySet()` gives key–value entries. When you need both a key and its value, iterate through `entrySet()` rather than repeatedly looking up each key.

```
for (Map.Entry<String, Integer> entry : grades.entrySet()) {
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}
```

3.3 HashMap, LinkedHashMap, TreeMap, Hashtable

Type	Ordering	Important detail
`HashMap`	None guaranteed	Allows one null key and null values
`LinkedHashMap`	Insertion order by default	Can be configured for access order
`TreeMap`	Sorted by key	Usually $O(\log n)$ operations
`Hashtable`	None guaranteed	Legacy synchronized class; disallows null keys/values

4 The equals/hashCode Connection

Sets and maps with hash-based implementations use the hash code to find a likely bucket and then equality to determine whether the key is already present. Equal objects *must* have equal hash codes, but distinct objects may collide. If you override `equals`, you must implement a consistent `hashCode` for hash-based collections to behave correctly.

```
import java.util.*;
class Student {
    final int id;
    Student(int id) { this.id = id; }
    @Override public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Student)) return false;
        return id == ((Student) obj).id;
    }
    @Override public int hashCode() {
        return Integer.hashCode(id);
    }
}
```

Avoid mutating fields that participate in `equals` or `hashCode` while an object is a key in a hash map or an element of a hash set. Doing so can make the object effectively unfindable.

5 A Complete Example: Course Enrollment

```
import java.util.*;
public class Enrollment {
    public static void main(String[] args) {
        Map<String, Set<String>> courses = new HashMap<>();
        courses.put("CMSC132", new HashSet<>());
        courses.get("CMSC132").add("Alice");
        courses.get("CMSC132").add("Bob");
        courses.get("CMSC132").add("Alice");
        System.out.println(courses.get("CMSC132").size()); // 2
        System.out.println(courses.get("CMSC132").contains("Bob"));
```

```
    }
}
```

This combines a map and a set: each course maps to a set of distinct enrolled students. A second registration by the same student does not increase the count.

6 Common Mistakes and Exam Traps

1. **Thinking `HashSet` preserves order.** It does not guarantee any particular iteration order.
2. **Thinking duplicate map values are forbidden.** Only keys must be unique.
3. **Calling `new Map<>()`.** `Map` is an interface; instantiate `HashMap` or another implementation.
4. **Assuming `put` always increases size.** Updating an existing key does not.
5. **Confusing `contains` and `containsKey`.** Sets use `contains`; maps use `containsKey`.
6. **Assuming `get` returning null proves a key is missing.** Check `containsKey` when null values are allowed.

7 Practice Questions with Answers

Q1. What is the size of a set after adding 7, 7, 8, 7? **Answer:** 2, because duplicates do not count.

Q2. What is the size of a map after `put("a",1)`, `put("b",1)`, `put("a",3)`? **Answer:** 2, and key a maps to 3.

Q3. Which implementation should you choose for automatically sorted unique integers? **Answer:** `TreeSet<Integer>`.

Q4. How can you count word frequencies efficiently? **Answer:** Use `HashMap<String,Integer>` and `getOrDefault`.

Q5. What is $\{1, 2, 3\} - \{2, 4\}$? **Answer:** $\{1, 3\}$.